Web Lab Practical: Basics of All 20 Questions

Bharati Vidyapeeth College of Engineering - Web Lab (2345117) - concept and steps for each question

Q1. Bootstrap registration form

Concept: Bootstrap is a ready-made set of styles, so the form looks good on mobile and desktop. JavaScript checks what the user typed.

Steps:

1. Link Bootstrap.
2. Make four inputs: name, email, mobile, password.
3. On submit, stop the page reload.
4. Check that no field is empty and that the email looks like a@b.com.
5. Show a success message.

Q2. Bootstrap login form

Concept: Same as Q1, with fewer fields.

Steps:

1. Two inputs and a button.
2. On submit, check that both have text.
3. Show an error if empty, otherwise show Welcome.

Q3. ES6 student marks

Concept: An array holds a list. filter() picks the items that pass a test. map() changes each item into something else.

Steps:

1. Make an array of students with name and marks.
2. filter those with marks of 75 or more.
3. map the result to just the names.
4. Print them.

Q4. ES6 product filter

Concept: Same as Q3, with two conditions.

Steps:

1. Make an array of products.
2. filter where the category matches and the price is below 1000.
3. map to the names.

Q5. React student profile

Concept: A component is a reusable piece of screen. Props are values the parent gives it, like function arguments.

Steps:

1. Write a Profile function that takes name, roll and course.
2. Show them inside a box.
3. Use the Profile tag with those values in App.

Q6. React counter

Concept: State is data the component remembers. When it changes, the screen updates.

Steps:

1. const [count, setCount] = useState(0).
2. Three buttons that call setCount to add 1, subtract 1, or set 0.

Q7. React routing

Concept: A single-page application changes the content without reloading the browser. React Router decides which component to show for each URL.

Steps:

1. Install react-router-dom.
2. Wrap the app in BrowserRouter.
3. Make Home, About and Contact components.
4. Add Link and Route for each.

Q8. React feedback form

Concept: A controlled form: each input's value lives in state.

Steps:

1. Three useState values, one per field.
2. onChange updates them.
3. On submit, save what was typed so it can be shown, then reset the three values to empty.

Q9. MongoDB student CRUD

Concept: MongoDB stores data as documents in collections. CRUD means Create, Read, Update, Delete.

Steps:

1. In mongosh, insertMany three students.
2. find() to show all.
3. updateOne with $set to change marks.
4. deleteOne using the _id.

Q10. MongoDB inventory CRUD

Concept: Same as Q9, with a condition.

Steps:

1. Insert three items.
2. find({ quantity: { $lt: 10 } }), where $lt means less than.
3. Update one quantity.
4. Delete one item.

Q11. Express and Mongoose employee app

Concept: Express is the server. Mongoose connects it to MongoDB. A schema describes what an employee document looks like.

Steps:

1. Install express and mongoose.
2. Connect to MongoDB.
3. Define the schema and model.
4. A GET route that returns Employee.find().

Q12. Express and Mongoose product app

Concept: Same as Q11, plus adding data.

Steps:

1. Product schema and model.
2. app.use(express.json()) so the server can read JSON.
3. A POST route that saves a product.
4. A GET route that shows all products.

Q13. Node.js file operations

Concept: Node's fs module reads and writes files. A Buffer is raw bytes.

Steps:

1. Write a file with three books.
2. Read it.
3. Append a fourth.
4. Read and print the final content.

Q14. Node.js HTTP server and events

Concept: Node can be a web server without Express. An EventEmitter lets you announce an event and react to it.

Steps:

1. http.createServer.
2. Check req.url: /home, /contact, or else 404.
3. Emit an event on each request and log it.

Q15. REST API for students (with Postman)

Concept: REST maps actions to HTTP methods: GET reads, POST creates, PUT updates, DELETE removes. Postman lets you test the API without a frontend.

Steps:

1. Keep students in an array.
2. Write four routes.
3. In Postman, send each request. For POST and PUT, put JSON in Body, then raw.

Q16. REST API for products

Concept: Same as Q15, with one more route that reads one item by its ID from the URL.

Steps:

1. GET all, GET one with /:id, and POST.
2. Test each in Postman.

Q17. MERN to-do application

Concept: MERN is MongoDB, Express, React and Node together. React is what the user sees, Express is the API, and MongoDB stores the tasks.

Steps:

1. Backend: a Task model, GET and POST routes, and cors().
2. Frontend: load tasks with useEffect and axios, and add a task from a form.
3. Run both servers.

Q18. MERN feedback application

Concept: Same pattern as Q17 with a feedback form.

Steps:

1. Backend: a Feedback model and routes to save and list feedback.
2. Frontend: a form that posts it, and a list of saved feedback.

Q19. Deploy MERN notes app

Concept: Deploying puts your app on the internet. Each part goes on a suitable service, and environment variables tell them where to find each other.

Steps:

1. Database on MongoDB Atlas.
2. Backend on Render, with MONGO_URI.
3. Frontend on Vercel, with VITE_API_URL pointing to the backend.
4. Add a note, refresh, and check it is still there.

Q20. Deploy MERN task manager

Concept: Same as Q19, but the app also updates and deletes tasks.

Steps:

1. Add PUT and DELETE routes.
2. Deploy as in Q19.
3. Test create, update and delete on the live site.

Pattern to remember for any MERN question

React form, then axios request, then Express route, then Mongoose model, then MongoDB. The response travels back the same way to React.